

Reproducibility of Zero-Shot LLM Pointwise Ranking with Global Context Aggregation

CS 5480 Deep Learning – Project Proposal

Utshab Kumar Ghosh, Christopher Elam, Ethan Garthe, Tristan Fox

Department of Computer Science

Missouri University of Science and Technology

{u.ghosh, cjefed4, ebgqf6, tfmwn}@mst.edu

I. PROBLEM STATEMENT

Recent work has shown that Large Language Models (LLMs) can rank documents by relevance in a zero-shot setting, without any training on labeled ranking data [1], [2]. There are two main ways to do this. Comparative methods ask the LLM to directly compare documents, either pairwise or listwise. These work well but are slow because they need many LLM calls. Pointwise methods score each document on its own, which is fast and easy to parallelize, but the scores tend to be inconsistent because the model never sees how documents relate to each other.

Long et al. [3] proposed a way to get the best of both worlds. Their method, called GCCP (Global-Consistent Comparative Pointwise Ranking), adds an “anchor” document to each pointwise prompt. This anchor is a query-focused summary built from the top candidate documents using spectral-based multi-document summarization, and it gives the model a shared reference point when scoring. They also propose PAGC (Post-Aggregation with Global Context), which combines GCCP scores with regular pointwise scores through a weighted linear aggregation. The result is a ranking method that is almost as effective as listwise approaches but runs at pointwise speed, and it needs no training at all.

We plan to reproduce and extend their work. Specifically, we will: (1) reproduce their reported results on standard IR benchmarks, (2) test whether the method works with LLM families the authors did not try (LLaMA-3, Qwen-2.5), and (3) run ablations on how the anchor document is built and how sensitive the method is to the quality of initial retrieval. Everything in this project involves inference-time prompting of Transformer-based language models on a clearly defined ranking task with standard metrics, which fits the scope of this course well.

II. RELATED WORK

Listwise and pairwise LLM ranking. Sun et al. [1] showed that ChatGPT and GPT-4 can rerank passages in a listwise fashion using a sliding window over candidate documents. Qin et al. [2] found that asking an LLM to compare two documents at a time (pairwise prompting) is very effective, but it requires $O(n^2)$ inferences, which gets expensive fast. Zhuang et al. [4] introduced setwise prompting as a middle ground, comparing

small groups of documents at once. Their experiments showed it hits a good balance between speed and quality. Pradeep et al. [5] built RankZephyr, an open-source model fine-tuned specifically for listwise reranking, and showed it can match GPT-4 on some benchmarks.

Pointwise scoring. Zhuang et al. [6] moved beyond simple Yes/No relevance prompts and explored fine-grained relevance labels, which improved pointwise scoring precision. Sachan et al. [7] took a different angle: they used the probability that an LLM would generate the query given a document as the relevance signal. Nogueira et al. [8] introduced monoT5, which treats reranking as a text-to-text problem. All of these are efficient, but because each document is scored in isolation, the model has no way to compare candidates against each other.

GCCP. Long et al. [3] tackle this isolation problem by inserting an anchor document into each pointwise prompt. The anchor captures the “gist” of what a good answer looks like for the query, so the model can implicitly compare each candidate against it. They tested this on Flan-T5 and Flan-UL2 models only, which leaves open the question of whether the idea transfers to decoder-only LLMs like LLaMA or Qwen.

III. PROPOSED APPROACH

We break the project into three phases.

Phase 1: Reproduce the original results. We will use the authors’ public codebase¹ to run BM25 retrieval (via Pyserini [9]), then apply the RG-YN pointwise baseline, GCCP, and PAGC on TREC DL 2019, DL 2020, and the BEIR benchmark. We will use the same models the authors used: Flan-T5-Large (~780M), Flan-T5-XL (~3B), and Flan-UL2 (~20B). Note that Flan-UL2 is a distinct model from Flan-T5-XXL. It uses a different training paradigm (mixture of denoisers) and was the largest model evaluated in the original paper. We will check whether our NDCG@10 numbers land within 1–2% of theirs.

Phase 2: Test on new LLM backbones. The original paper only covers encoder-decoder models from the Flan family. We want to know if GCCP still helps when you swap in a decoder-only model. We plan to try LLaMA-3.1-8B-Instruct,

¹<https://github.com/ChainsawM/GCCP>

Qwen-2.5-7B-Instruct, and possibly Phi-3-mini (3.8B). This will require adapting the prompt format, since decoder-only models handle instructions differently than encoder-decoder models. If GCCP works across architectures, that strengthens the paper’s claims considerably. If it doesn’t, that’s an equally interesting finding.

Phase 3: Ablations. Three questions we want to answer:

- *Does the anchor construction matter?* The authors build a query-focused summary as the anchor using spectral-based multi-document summarization. We will compare this against simpler alternatives: using the top-1 BM25 document, a random candidate, or an LLM-generated “ideal answer” to the query. The original paper already tests these strategies (Table 5), so this part is primarily reproduction and verification.
- *Does initial retrieval quality matter?* The paper uses BM25 for first-stage retrieval. As a **novel extension** beyond the original paper, we will swap in a dense retriever (E5-base-v2) to see if GCCP is sensitive to how the initial candidate set is constructed. This experiment does not appear in the original work.
- *How sensitive is the score aggregation?* The original paper combines scores using a linear scheme (Eq. 11): $f_{\text{final}}(q, d_i) = \frac{1}{|R|+1} (\sum_R f(q, d_i) + f_c(q, d_i, d_a))$, where R is the set of independent pointwise methods. They also tested several rank aggregation methods (Borda, Condorcet, Copeland, etc.) in Section 4.4.1 and found them all roughly equivalent. We will verify this finding and additionally explore whether non-uniform weighting between the GCCP and pointwise components can improve results, which the original paper did not test.

None of these phases require any model training or fine-tuning. The entire project is inference-only.

IV. DATASETS

We use publicly available, standard IR benchmarks. No new data collection is needed.

TREC Deep Learning Track [10]: DL 2019 has 43 queries and DL 2020 has 54 queries, both with graded passage-level relevance judgments over the MS MARCO v1 passage corpus [11] (about 8.8 million passages). BM25 top-100 results are generated using Pyserini’s pre-built indices, and the authors’ repository includes ready-to-run shell scripts for this.

BEIR [12]: A benchmark covering many different domains and query types. Following the original paper, we will evaluate on 8 subsets: Covid, Touche, DBPedia, SciFact, Signal, News, Robust04, and NFCorpus.

V. EVALUATION PLAN

Metrics. NDCG@10 is the primary metric, consistent with the original paper and standard practice in passage ranking. As an addition beyond what the original paper reports, we will also include Precision@10, Recall@10, and NDCG@100 to give a more complete picture of ranking quality at different cutoffs.

Baselines. We compare five configurations: (1) BM25 alone, (2) RG-YN pointwise re-ranking [3], (3) GCCP, (4) PAGC (GCCP + pointwise scores aggregated), and (5) a listwise RankGPT-style baseline [1] if compute allows. This gives us the full spectrum from no re-ranking, through pointwise, to listwise.

Setup. BM25 retrieves 100 candidates per query. All re-ranking runs on this candidate set using the same prompt templates from the released code. We will use paired t-tests ($p < 0.05$) for statistical significance.

Hardware. We have $2 \times$ NVIDIA Ada A6000 GPUs (48GB each) and access to the university’s Mill cluster. Flan-T5-XL (3B params) needs roughly 6GB in FP16 and runs comfortably on a single GPU. Flan-UL2 (20B) needs about 40GB for weights alone in FP16. The original authors used $4 \times$ A6000 GPUs for their experiments, so for UL2 we will either split the model across both A6000s or use the Mill cluster. Decoder-only 7–8B models need around 16GB and fit easily on a single A6000.

VI. TIMELINE

Week	Milestone
6–7	Set up environment, run BM25 retrieval, reproduce the RG-YN pointwise baseline
8–9	Reproduce GCCP and PAGC, compare our numbers to the published results, debug any discrepancies
10–11	Run generalization experiments with LLaMA-3, Qwen-2.5, Phi-3
12	Ablation experiments on anchor strategy, dense retrieval, aggregation
13	Finish BEIR experiments, put all results together
14	Write up analysis, make figures, start the final report
15	Finish the report, prepare the poster

VII. TEAM RESPONSIBILITIES

We have not finalized the exact division of labor yet, but here is our tentative plan:

- **Member A:** Environment setup, BM25 retrieval pipeline, reproduction of baselines on TREC DL.
- **Member B:** GCCP/PAGC reproduction and verification against published numbers.
- **Member C:** Generalization experiments with new LLM backbones, prompt adaptation for decoder-only models.
- **Member D:** Ablation experiments (anchor strategies, dense retrieval, aggregation methods), BEIR evaluation, figures and tables.

Everyone will contribute to the final report and poster. We will finalize the assignment once we have a clearer picture of each member’s availability and strengths.

REFERENCES

- [1] W. Sun, L. Yan, X. Ma, S. Wang, P. Ren, Z. Chen, D. Yin, and Z. Ren, “Is chatgpt good at search? investigating large language models as re-ranking agents,” in *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023, pp. 14918–14937.

- [2] Z. Qin, R. Jagerman, K. Hui, H. Zhuang, J. Wu, L. Yan, J. Shen, T. Liu, J. Liu, D. Metzler *et al.*, “Large language models are effective text rankers with pairwise ranking prompting,” in *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024, pp. 1504–1518.
- [3] K. Long, S. Li, C. Xu, J. Tang, and T. Wang, “Precise zero-shot pointwise ranking with llms through post-aggregated global context information,” in *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2025, pp. 2384–2394.
- [4] S. Zhuang, H. Zhuang, B. Koopman, and G. Zuccon, “A setwise approach for effective and highly efficient zero-shot ranking with large language models,” in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 38–47.
- [5] R. Pradeep, S. Sharifymoghaddam, and J. Lin, “Rankzephyr: Effective and robust zero-shot listwise reranking is a breeze!” *arXiv preprint arXiv:2312.02724*, 2023.
- [6] H. Zhuang, Z. Qin, K. Hui, J. Wu, L. Yan, X. Wang, and M. Bendersky, “Beyond yes and no: Improving zero-shot llm rankers via scoring fine-grained relevance labels,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, 2024, pp. 358–370.
- [7] D. Sachan, M. Lewis, M. Joshi, A. Aghajanyan, W.-t. Yih, J. Pineau, and L. Zettlemoyer, “Improving passage retrieval with zero-shot question generation,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022, pp. 3781–3797.
- [8] R. Nogueira, Z. Jiang, R. Pradeep, and J. Lin, “Document ranking with a pretrained sequence-to-sequence model,” in *Findings of the association for computational linguistics: EMNLP 2020*, 2020, pp. 708–718.
- [9] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Pradeep, and R. Nogueira, “Pyserini: A python toolkit for reproducible information retrieval research with sparse and dense representations,” in *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*, 2021, pp. 2356–2362.
- [10] N. Craswell, B. Mitra, E. Yilmaz, D. Campos, and E. M. Voorhees, “Overview of the TREC 2019 deep learning track,” *arXiv preprint arXiv:2003.07820*, 2020.
- [11] P. Bajaj, D. Campos, N. Craswell, L. Deng, J. Gao, X. Liu, R. Majumder, A. McNamara, B. Mitra, T. Nguyen *et al.*, “Ms marco: A human generated machine reading comprehension dataset,” *arXiv preprint arXiv:1611.09268*, 2016.
- [12] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, “Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models,” *arXiv preprint arXiv:2104.08663*, 2021.